Name : Akansha
Rollno : 2401010195
Program : B.Tech CSE core (A)
Subject : DataBase Management System
Sub. Code : ENCS 204
Submitted to : Miss Mansi Kajal

Section - A

Q What is transaction in DBMS?

A transaction in DBMS is a logical unit of work that consists of one or more database operations such as Insert, UPDATE, DELECT, SELECT. These operations must be executed completely or not at all.

→ A transaction ensures that the database remains consistent even if something goes ~~away~~ wrong.

Ex- Bank transfer :-

- Debit ₹ 500 from Account A.
- Credit ₹ 500 to Account B.

→ Both steps must happen together. If one fails, the whole transaction is rolled back.

Key Points -
- Begin with BEGIN TRANSACTION
- Ends with COMMIT (save) or ROLLBACK (undo)
- Maintains data integrity.

Q Explain ACID properties.

ACID properties ensure reliable & Style transaction processing.

→ A → Atomicity (All or Nothing)

- Either all operations execute or none
- If one fails → entire transaction is rolled back.

Eg - If money is debited but not credit → Rollback

→ C → Consistency (Valid State)

- Database must move from one valid state to another.
- Rules, constraints & integrity must be maintained.

Ex- Account balance can not bcz -ve if not allowed.

→ I → Isolation (No Interface)

- Transaction execute independently
- Intermediate states are not visible to other transactions.

Ex- Two users updating same account should not conflict.

→ D → Durability (Permanent Save)

- Once committed, data is permanently stored.
- Even System crash can not lose committed data.

Ex- After payment success, data remains saved.

Q What is concurrency control?

Concurrency control is the process of managing multiple transactions executing at the same time without causing data inconsistency.

→ It ensures that results are correct & conflict - free.

→ Problems without concurrency control

- Lost Update
- Dirty Read
- Unrepeatable Read

→ Techniques Used—

- Locking (Shared & Exclusive locks)
- Timestamp ordering.
- multiversion Concurrency Control (MVCC)

Ex- Two users editing same bank balance simultanesouly system must have handle properly.

Q What is deadlock?

A deadlock occurs when two or more transactions are waiting for each other indefinitely & none can proceed.

→ It creates a cycle of waiting.

Ex- Transcation of T_1 locks resource A & wait for B.
Transaction of T_2 locks resource B & wait for A.
→ Both stack forever.

Solutions- Deatlock prevention (avoid conditions)
Deadlock detection (find & revoke)
Deadlock recovery (rollback one transaction)

Q What is Serializability?

Ans- Serialzability ensures that the result of concurrent transactions is the same as if they were executed one after another.
→ It is the highest level of correctness in concurrency control.

Types -
- Conflict Serializability
- View Serializability

Example- If T_1 & T_2 runs together, result should match either.

$T_1 \rightarrow T_2$
or
$T_2 \rightarrow T_1$

Q What is recovery in DBMS?

Recovery is the process of restoring the database to a correct state after failure (like crash, power failure, system error)
→ Ensures data is not lost.

Techniques -
- Log-based recovery
- Check points
- Backup & restore

E.g- If System crashes after money is debited but before credit → recovery restores correct state.

Q What is Check Point?

A Check-point is a mechanism where DBMS saves current state of the dB to stable storage.

→ It reduces recovery time.

How it works-

- DBMS writes all change to disk
- Marks a checkpoint in log file
- During recovery - starts from checkpoint instead of beginning.

Ex- Instead of reprocessing all transactions, system restarts from last checkpoint.

Section-B

Q Explain ACID properties using Banking Example.

Consider a bank transfer of ₹1000 from Account A to Account B.

Atomicity (All or nothing)

- Either both operations happen:
  - Debit from A
  - Credit to B
- Or one happen-
  - If systems crashes after debiting A but before crediting B → transaction is rolled back.

Consistency (Valid State)

→ Before transaction: Total money = A+B

→ After transaction: Total money should remain Same

→ If A had ₹5000 & B had ₹2000 → total ₹7000

→ After transaction → Still ₹7000 (no data corruption)

Isolation (No Interface)

→ If two users access Same account simultaneously.

→ One transaction should not see intermediate state.

→ User 1 transferring money & User 2 checking balance → User 2 Should not see half-updated balance.

Durability (Permanent Storage)

→ Once transaction is committed:

→ Data is permanently saved.

→ Even if System crashes after successful transfer → updated balances remain safe / secure.

Q Illustrate a Deadlock Situation with Two transactions.

Two transaction T_1 & T_2 & two resources: Account A & Account B.

→ Execution - T_1 locks Account A

T_2 locks Account B

Now - T_1 wants to lock B → waits for T_2

T_2 wants to lock A → waits for T_1

→ Both are waiting for each other → Deadlock occurs

T_1 → holds A, waiting for B.

T_2 → holds B, waiting for A.

Locks are used to prevent conflicts.

- Shared lock (Read)
- Exclusive lock (Write)

→ Deadlock happens due to improper locking order.

How to Handle Deadlock

- Prevention (avoid circular wait)
- Detection (wait for graph)
- Recovery (Rollback one transaction)

Q Explain Serial Execution Vs Concurrent Execution with Example.

Serial Execution – Transactions execute one after another.

Ex – T_1 : Read A Write A

T_2 : Read A Write A

- First T_1 Complete then T_2 Starts
- No Conflicts — Easy to Manage — Slow performance

Concurrent Execution – Transaction execute Simulatensouly

Eg- T_1 : Read A → May cause problem like :
T_2 : Read A → Lost Update
T_1 : write A → Dirty Read
T_2 : write A

Feature	Serial Execution	Concurrent Execution
Speed	Slow	Fast
Safety	High	Needs Control
Complexity	Simple	Complex
Data Issue	No	Possible

Section-C

Q. A banking system transfers money b/w accounts. Explain how ACID properties apply.
Consider a transaction: Transfer ₹1000 From Account A to B.

Atomicity – Both operations must occur:-
i) Debit ₹ 1000 From A ii) Credit ₹ 1000 to B.
If one fails → entire transaction is Rolled back, ensures no partial update.

Consistency – Total balance must remain constant
↳ DB rules (like min balance) must be maintained.
Eg- A+B total remain unchanged after transfer.

Isolation – Multiple transaction should not interfere
- Intermediate State (like money deducted but not added) is not visible.
other users see either old or new balance, not half-done transaction.

Durability – Once transaction is committed
Changes are permanently Saved.
Even after system crash, updated balance remain Safe

Q Two transactions update same data simultaneously. Identify problem

When two transactions run at the same time on same data, concurrency problems may occur:

1) Lost - Update Problem :- One transaction overwrites another update

Ex- T_1 reads balance = ₹1000

T_2 reads balance = ₹1000

T_1 updates = ₹1200

T_2 updates = ₹1500

Final value = ₹1500 (T_1 update lost)

2) Dirty Read :- One transaction reads uncommitted data.

Ex- T_1 ~~reads~~ updates balance but not committed

T_2 reads that value

T_2 rolls back

→ T_2 used invalid data.

3) Unrepeatable Read :- Same query gives different results.

Ex- T_1 reads balance = ₹1000

T_2 reads balance to ₹2000

T_1 read again = ₹2000

4) Phantom Read → New rows appear during transaction.

Q A System crashes during transaction. Explain Recovery steps.

When a system crashes, DBMS uses recovery mechanism to restore data.

Step ① - Use log file

- DBMS maintains a log file with all operations.

↳ Short Transaction → Update → Commit / Rollback

Step ② - Identify Transaction State

- Committed transactions → must be REDO.
- Uncommitted transaction → must be UNDO.

Step ③ - Perform Recovery

REDO (Repeat) • Reapply committed transaction → Ensures durability.

Step ④ - UNDO (Rollback)

- Reverse incomplete transactions → Ensures atomicity.

Step ⑤ - Check Point (if available)

→ Start recovery from last checkpoint

→ Reduce recovery time.

Q Draw a transaction timeline & Identify conflict.

Timeline example -

Time T_1 : R(A) - - - W(A)

T_2 : R(A) - - - W(A)

T_1 reads A

Before T_1 writes, T_2 also read A

Both Update A independently.

Final value may overwrite one transactions.

Q Explain Recovery technique.

Log-Based Recovery

- DBMS keeps a log file that records every transaction step.

→ Log contains - • transaction start
• Data changes
• commit / Rollback

→ During Crash -

committed transactions → REDO

Uncommitted transactions → UNDO

Ex- T_1 : Start → Update → commit

T_2 : Start → Update (crash)

→ After Recovery - T_1 → REDO

T_2 → UNDO

→ Log Based Recovery ensures -

→ Data Consistency

→ No partial Updates

→ Fast restoration after failure.